

UID $\rightarrow$ 28BCS10104

KRG-2B

Name $\rightarrow$ Paramjit

competitive coding

[Experiment-7]

Algorithm $\rightarrow$ 1.) create a Adjacency list from the given

② Initialize a queue (src, cost, stops)

& a Distance array $\rightarrow$ dist[] = ∞ , dist[src] = 0

③ while queue is not empty :-

- Extract (node, cost, stops)

- if stops > K, skip

- for each neighbour :

 - if new cost is smaller :

 - update distance

 - Push (neighbour, new cost, stops + 1)

④ if destination is unreachable, return -1

⑤ otherwise return dist[dest]

code $\{$

```
vector<vector<pair<int,int>>> adj(n);
```

```
for(auto &f : flights){
```

```
    adj[f[0]].push_back({f[1], f[2]});
```

```
}
```

```
queue<tuple<int,int,int>> q;
```

```
q.push({src, 0, 0});
```

```
vector<int> dist(n, INT_MAX);
```

```
dist[src] = 0;
```

```

while (!q.empty()) {
    auto [node, cost, stops] = q.front();
    q.pop();
    if (stops > k) continue;
    for (auto [adjNode, price] : adj[node]) {
        if (cost + price < dist[adjNode] && stops <= k) {
            dist[adjNode] = cost + price;
            q.push({ adjNode, dist[adjNode], stops+1 });
        }
    }
}

if (dist[src] == INT_MAX) return -1;
return dist[dst];
}
}

```

Time complexity $\rightarrow$ As we are using BFS (with k stops)

$$\rightarrow O(K \times E)$$

Space complexity $\rightarrow O(N)$